flint-forge — architecture

A git server per repository with S3 as its only durable state, drawn as three planes: the **data plane** every byte of a clone or push crosses, the **durable path** that alone can write the bucket, and the **control plane** that places, admits and keeps the server alive. Each component sits on the planes it belongs to and no others — and the runner that replaced nginx and fcgiwrap sits on the data plane alone.

Scope. The architecture of flint-forge as built and drilled through 2026-09-05: the components, the flows between them hop by hop, the transaction that makes an acknowledged push durable, the single-writer lease and its takeover, the operator and the door, the boundaries the planes draw, the campaigns that stand behind every number, and where forge sits beside its prior art. It is a description of what exists, with what was measured and what was refuted; it proposes nothing.

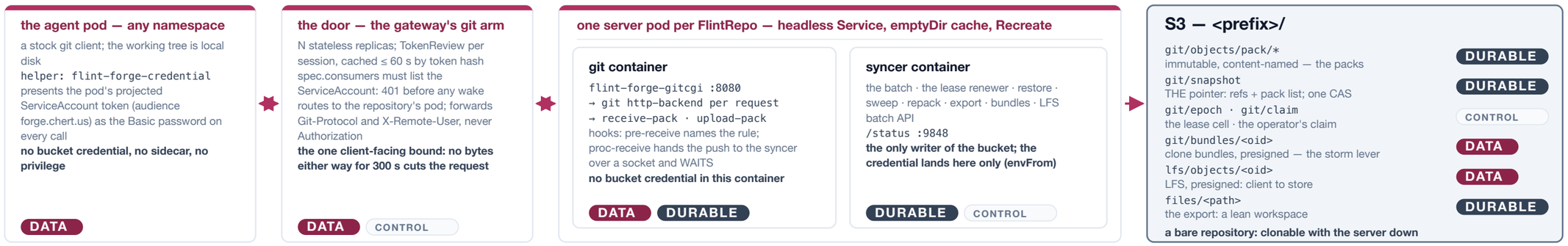
Sources of record. `docs/plans/flint-forge-design.md` (the design, its decisions and its review), `docs/plans/flint-forge-simplification-2026-09-05.md` (the party table, the defect list X1–X12, the runner), `formal/ForgeSync.tla` (the theorems and the mutations), `forge/e2e/scale/README.md` (runs 1–7 on real S3), `forge/e2e/latency/`, `forge/e2e/composition/`, `forge/e2e/run-rights.sh`, and the code under `forge/syncer/`, `spdk-csi-driver/src/forge_operator/` and `spdk-csi-driver/src/lite_gateway/git.rs`. Where this document and the design disagree, the design's dated corrections win; where it and the code disagree, the code wins and the document is wrong.

Regenerating. `docs/architecture/forge/forge-diagrams.py` writes every plate with the family deck's kit (`docs/architecture/diagrams.py`); `docs/architecture/forge/build.sh` validates, measures every label against its box in Chrome, renders the PDF and extracts the Markdown. Diagrams and the Markdown are built, never edited by hand.

Revision. Tree `be76cc9c` and its successors of 2026-09-05: the TLA+ model with the two rotation fixes it found, the `flint-forge-gitcgi` runner accepted on cluster runby, the concurrent-clone leg. Nothing in this document has served a team; every number is a rig's, and the last page says what that leaves out.

The three planes

Forge is stock git in a pod, a stateless door in front of it, one syncer beside it, and a bucket behind it. The same components look different depending on which question is asked of them — where do the bytes go, what can change what a restore sees, who decides that the pod exists — and those three questions are the three planes. A component can be on more than one plane; the plate's matrix says which, and the point of drawing it is that the answers do not overlap where it matters: the process that parses untrusted HTTP is not the process that can write the bucket, and the thing that places the pod never reads what the pod stores.



plane · component	agent git	credential helper	the door	gitcgi runner	http-backend, receive-pack, upload-pack	pre-receive, proc-receive	the syncer	packs, snapshot	epoch, claim	operator, FlintRepo, apiserver
data plane every byte of a clone or push	●	●	●	●	●	●	–	–	–	–
durable path what can write the bucket	–	–	–	–	–	●	●	●	–	–
control plane placement, admission, the lease	–	–	○	–	–	–	○	–	○	○

The data plane — every byte, nothing durable

A clone, fetch or push is the smart protocol from a stock client to stock git: the door proxies it, the runner execs git http-backend per request and streams both directions as they arrive, receive-pack and upload-pack do the git.
A 40 GiB push crosses the runner twice: the pack in, the sideband keepalives and the report out. That is why fcgiwrap's buffering mattered: it held the keepalives that keep a long push alive.
One client-facing bound, the door's 300 s inactivity cut. The runner has no timeout of its own and answers 503 past a declared ceiling rather than queueing (run 7: five pushes and eight clones at once, a 70 s stall, all served).

The durable path — what can write the bucket

proc-receive hands the push to the syncer and waits. The syncer judges it under the agreed view, renews the lease once, uploads every complete pack as immutable content-named objects, CASes ONE snapshot naming the refs and the packs, applies update-ref, and only THEN reports ok.
The pack is verified by git, uploaded by the syncer, named by the CAS; the runner implements no git, holds no credential and never touches S3.
The process that parses untrusted HTTP is not the process that can write the bucket. Machine-checked: told ok ⇒ durable; every landed pack complete; no straggler lands after a restore.

The control plane — placement, admission, liveness

The operator turns a FlintRepo into a ConfigMap, a headless Service, a one-pod Deployment and a NetworkPolicy, polls the pod's own /status for its phase, and parks an idle repository at replicas 0.
The door admits by TokenReview and consumers, routes by repository, and wakes a parked one — 401 before any wake, the request held up to 180 s on the CR, never the pod.
The lease cell is the writer's coordination: a token that must keep moving, six quiet polls before a takeover, a rotation of the snapshot on every claim.
The operator never reads the bucket, and A3 touched none of this: chart, CRD, door and rigs did not change.

Reading the placement

The runner (A3) is squarely on the data plane and nowhere else. It replaced nginx and fcgiwrap in the git container after the runbx drill found three of its four front-layer defects in those two processes — buffered keepalives, two 60 s client timeouts nobody had listed, a four-worker ceiling that queued in silence — and it removed that class rather than tuning it. Nothing on the durable path or the control plane moved, which is why the chart, the CRD, the door's URL formula and the rigs did not change.
Two boundaries are drawn by the planes. The bucket credential exists in the syncer container only, so a compromise of the HTTP-facing process cannot write S3. And X-Remote-User is the principal only behind the NetworkPolicy that admits the door's pods alone: reached directly, the git port believes the header (measured, run-rights.sh: with the policy deleted a forged header merges into main).

durable path · data plane · control · hazard / trust boundary

The data plane is every byte and nothing durable. A clone, fetch or push is the ordinary smart protocol from a stock client to stock git. The agent's credential helper presents the pod's own projected ServiceAccount token as the Basic password; the door verifies it and routes; the runner in the git container execs `git http-backend` once per request and streams the request body into it and its output back out as each arrives; `receive-pack` and `upload-pack` do the git. Nothing on this plane holds a bucket credential and nothing on it is durable: a pack received is on local disk, which is a cache, until the durable path has named it. The runner (`flint-forge-gitcgi`, option A3 of the simplification note) is on this plane alone. It replaced nginx and fcgiwrap after the runbx drill found three of its four front-layer defects in those two processes — buffered keepalives that cut a 40 GiB push 311 s into its hook wait, two 60 s client-side timeouts nobody had listed, a four-worker ceiling that queued in silence — and because it implements no git and touches no bucket, replacing it moved nothing else: the chart, the CRD, the door's URL formula and every rig were unchanged.

The durable path is what can write the bucket, and it is one process. `proc-receive` is the only place the data plane hands anything to the writer, and it hands a request, never a credential: it sends the push's commands and its pack's name to the syncer over a Unix socket and waits. The syncer judges, renews the lease once, uploads every complete pack as an immutable content-named object, CASes one snapshot naming the refs and the packs, applies the local ref transaction and only then reports `ok`. The credential exists in the syncer container only, by `envFrom`. Page 3 draws the transaction and the theorems that hold over it.

The control plane is placement, admission and liveness, and it never reads the bucket. The operator turns a `FlintRepo` into a ConfigMap, a headless Service, a one-pod Deployment and a NetworkPolicy, polls the pod's own `/status`, and parks an idle repository at replicas 0. The door admits by TokenReview and `spec.consumers`, routes by repository, and wakes a parked one by annotating the CR and waiting on it. The lease cell in S3 is the writer's coordination — a token that must keep moving, six quiet polls before a takeover, a rotation of the snapshot on every claim — and it is placed on this plane because it decides who may write, not what is written; the fence it produces is what the durable path relies on. Two boundaries fall out of the placement: the credential boundary between the two containers of one pod, and the principal boundary, where `X-Remote-User` means something only behind the NetworkPolicy that admits the door's pods alone. Page 6 says what each was measured to hold.

The data plane

Two flows cross the same six hops: a clone or fetch, which is a request and a pack coming back, and a push, which is a pack going in and a sideband coming back for as long as the hooks take. The plate names what crosses each hop and what each party is allowed to do to the flow. The list of parties matters because the campaign kept finding defects in members of it that nobody had written down; after the runner the list is what it claims to be, and the last row shows the control arm failing each leg the runner passes.

Clone / fetch — what crosses each hop

agent pod · stock git	the door	gitcgi runner	git http-backend	receive-pack / upload-pack	emptyDir cache
GET info/refs?service=git-upload-pack, then POST git-upload-pack: ls-refs with a prefix (protocol v2), fetch `Basic x:<SA token>`	TokenReview → 401 or forward; consumers; the wake if parked `X-Remote-User · Git-Protocol` HTTP, cleartext in-tree	execs http-backend per request with the CGI environment from the request; streams the request body in and stdout out `REMOTE_USER · GIT_PROTOCOL`	stock git's CGI: negotiates, forwards to upload-pack; sets no policy `GIT_PROJECT_ROOT · EXPORT_ALL`	upload-pack with bitmaps; advertises a presigned bundle URI when the client opted in `uploadpack.advertiseBundleURIs`	the packs the snapshot named, restored at start; a clone reads local NVMe `objects/pack/*.pack .idx .bitmap`

Push — the pack in, the sideband out

agent pod · stock git	the door	gitcgi runner	git http-backend	receive-pack / upload-pack	emptyDir cache
POST git-receive-pack: the commands, then the pack, chunked; reads sideband packets until the report **a 40 GiB push is one request of ~19 minutes**	the inactivity clock is touched by every chunk either way; 300 s of silence cuts the request **the one client-facing bound, by design**	stdin from the request body as it arrives; stdout to the client as it is written; the child killed when the client goes **no timeout, no body limit; 503 past the ceiling (64)**	execs receive-pack --stateless-rpc; index-pack writes the pack into quarantine, then migrates .keep .pack .rev .idx **the index lands LAST — X1**	pre-receive: the policy, by principal. proc-receive: the push to the syncer, then WAIT `receive.keepAlive = 5 s` a keepalive every 5 s of hook silence	the pack is on local disk; the syncer reads it from here to upload it and names it only once its .idx exists **durable only after the snapshot CAS**

The parties between the client and git, and the knob each one has — after A3

git client	the door	the runner	receive-pack	the hook	the syncer	kubernetes
no inactivity bound unless http.lowSpeedLimit/Time is set (off by default); the rigs set only the token header	no bytes either way for upstreamTimeoutSecs (300) cuts the request; a wake holds a slot ≤ 180 s; TokenReview cached ≤ 60 s	none of its own: no body limit, no timeout; a ceiling of 64 concurrent requests that answers 503 at once. Removed: nginx's two 60 s defaults (X3) and fcgiwrap's four workers (X4)	receive.keepAlive pinned at 5 s in the syncer's config (X5): an empty sideband packet whenever the hooks have been quiet that long	none, correct as is: it waits for the syncer's report, and the process's exit closes its socket	no batch deadline; the wedge detector is the renewer, gated on progress: a batch that moves no bytes for 6 × 10 s is taken over	terminationGracePeriodSeconds 30 against batches of minutes; SIGTERM is seen between batches, so a roll mid-push is a SIGKILL at 30 s (X6, measured: the push is told failed, cleanly)

A stalled client — S5

1 GiB push, the client SIGSTOPped 3 s into the body for 70 s, then resumed.
runner: acknowledged, and the bucket holds it.
nginx control: HTTP 502 — client_body_timeout's 60 s default cut the client, the door relayed it.

Five pushes at once, eight clones at once — S6, S10

Five 256 MiB pushes stopped mid-body; an advertisement request timed beside them. Eight single-branch clones of a 1 GiB branch, a push into the storm.
runner: 5 receive-packs, the request answered in 0 s, 5/5 durable; 8 upload-packs at the peak, all 8 at the tip, 51 s wall against 16 s alone.
control: 4 receive-packs, the request queued 60 s, 1/5; 4 upload-packs, clones in pairs.

Keepalives through the front — S9, and its control S8

20 GiB push, a 232 s hook wait, every packet the client read stamped by git's own trace.
runner: 48 packets, the first 5.8 s after the pack, longest gap 5.8 s.
control: 49 packets in ONE burst with the report, 237.9 s after the pack.
S8: keepalive off and the door at 30 s — the connection closed 30.5 s after the upload; the batch landed anyway.

The storm lever — bundle URIs (F8, 2026-09-04)

A thousand clones are ~130 CPU-s but 43 GB from one NIC: egress binds first. A bundle is cut on a floor, uploaded beside the packs, advertised as a presigned URL.
1,000 clones: 5.7 MiB of server egress with the client opted in (transfer.bundleURI=true), 40,409 MiB without — 7,000×.
Gitea on the same corpus, 100 clones: 4,019 MiB, level with forge's control arms.

data plane — the control arm's failure — the class the runner removed

A push is one HTTP request that can last twenty minutes, and every party on the path has an opinion about silence. The client sends the commands and then the pack, chunked; the server answers nothing until the pack is in, then `receive-pack` runs the hooks and, while they are quiet, writes an empty sideband packet every `receive.keepAlive` seconds (pinned at 5 s in the syncer's config so the guarantee does not rest on git's default). The door's inactivity clock is touched by every chunk in either direction and cuts the request after 300 s of nothing; that is the one client-facing bound, by design, and it is why the keepalives must reach the door as they are written. Through fcgiwrap they did not: it held the CGI's output until the request ended unless a patch-specific parameter was set, and the 40 GiB push of run 3 was cut 311 s into its hook wait with two keepalives arriving in one burst with the report. Through the runner (run 7, S9) a 20 GiB push's 232 s hook wait carried 48 packets to the client with a longest gap of 5.8 s; through the nginx control (the same syncer, the last nginx image) the 49 packets arrived in one burst 237.9 s after the pack. The control with keepalives switched off and the door at 30 s was cut 30.5 s after the upload ended, which is the bound being real.

The runner has no knobs it does not declare. It sets no timeout and no body limit — the door owns the client-facing bound — and it holds a ceiling of 64 concurrent requests that answers 503 at once rather than queueing. The two knobs it removed were the class the drill kept finding: nginx's `client_body_timeout` and `send_timeout` defaulting to 60 s beside backend bounds of an hour (X3), and `FCGIWRAP_CHILDREN=4` (X4). Run 7 exercised each: a client SIGSTOPped 3 s into a 1 GiB pack for 70 s was acknowledged through the runner and got a 502 through the control; five 256 MiB pushes stopped mid-body held five `receive-pack` processes and an advertisement request beside them was answered in 0 s through the runner, where the control ran four and queued the request for 60 s; eight concurrent single-branch clones of a 1 GiB branch peaked at eight `upload-pack`s and all landed at the tip in 51 s against 16 s for one alone, where the control peaked at four and served them in pairs. A push launched into the middle of the clone storm was acknowledged in both arms.

What the data plane cannot do is take the server out of a storm; the bucket can. A thousand clones cost about 130 CPU-seconds but 43 GB from one pod's NIC, so egress binds long before CPU. The lever is git's own bundle URI: the syncer cuts a full bundle on a floor, uploads it beside the packs, lists it in the snapshot and advertises a presigned URL that `upload-pack` hands to a client that opted in (`transfer.bundleURI=true`, protocol v2, both sides ≥ 2.40 — three conditions the first draft missed, each verified). Measured on EC2 at 1,000 clones: 5.7 MiB of server egress with the opt-in, 40,409 MiB without. Gitea on the same corpus lands level with forge's control arms, within 0.5%, which is the honest statement of what a bought forge gives up. LFS takes the same shape: the batch API runs in the syncer and answers presigned URLs, so a checkpoint goes client-to-store and the pod sees a few hundred bytes of JSON.

The durable path

The review's central finding shaped this path: under `proc-receive`, git's `receive-pack` serialises nothing and checks nothing for the commands it hands off — no old-oid check, no fast-forward test — and two concurrent pushes to one ref run their hooks fully overlapped. So the hook does not sync; the syncer does, one process per repository holding the writer lock for every path to S3, and a push becomes one step in one batch that ends with one CAS. Everything that can change what a restore sees goes through steps 3 and 4, under one lock, in one process.

One batch, one transaction — receive-pack serialises nothing under proc-receive, so the syncer does

1 collect and judge
every pending push, in arrival order, against the local refs, the snapshot AND the syncer's running view; old-oid must match both or ng stale, fetch first; fast-forward and policy here, since receive-pack no longer checks them; a refs/for/<target> runs the merge and packs what it created

2 renew the lease, once
epoch_renew with If-Match on our token; a 412 takes the lost-response rule (re-read; adopt if the cell still names us) else it is the fence: ng to every hook, stop serving reads too, exit

3 hash and upload the packs
every local pack the snapshot does not name and whose .idx exists, with .idx .bitmap .rev: unconditional PUTs of content-named keys, siblings four in flight; multipart above 64 MiB, the CRC-64/NVME accumulated per part beside its PUT and judged by S3 at Complete

4 ONE snapshot CAS
If-Match on the etag this syncer last synced (If-None-Match:* when none): refs and the full pack list move as one object; a reader never sees half a batch. Under the lock a 412 can only be another server — the fence

5 the ref transaction
git update-ref --stdin applies every accepted ref locally as one transaction

6 THEN the report
ok or ng per ref to each waiting hook; per-ref ok reaches the client as it is emitted, so the report follows the whole transaction. A crash before this fails every push at the client; the restart restores from the snapshot

7 after the report
best effort, never a gate: derived info/packs then info/refs (in that order), the export, the sweep, a repack when the pack count passes the threshold

What S3 holds — <prefix>/

`git/objects/pack/pack-<sha>.pack`
immutable, content-named by git; with .idx .bitmap .rev beside it; unconditional PUT

`git/snapshot`
THE pointer: {seq, epoch, refs{name: oid}, packs[], bundles[], exported_commit}; CAS'd by etag; the only mutable object the server trusts

`git/epoch`
the single-writer lease cell: {epoch, token, holder, released}; every write a CAS

`git/claim`
the operator's project claim: a foreign projectId refuses the start

`git/objects/info/packs · git/info/refs · git/HEAD`
derived, for the dumb protocol and for humans; the server never reads them

`git/bundles/<oid>.bundle`
clone bundles, listed in the snapshot, advertised presigned

`lfs/objects/<oid>`
LFS objects, presigned both ways: the pod sees a few hundred bytes of JSON

`files/<path> + .flint/lean/`
the export: a valid lean workspace, published after the report by the shipped flint-sync barrier

Versioning OFF: nothing pins a version

The theorems — ForgeSync.tla, eight runs in the gate

Two syncers, two pushes, the batch at store-request granularity with a crash between any two steps, the renewer's sensor kept distinct from real movement, a challenger, a successor's claim, rotation, sweep and restore, git's .idx-last migration, a client that can hang up.

Invariants — strict run, 2,776,804 states, depth 52

Told ok ⇒ the ref landed in the snapshot and the pack is in the bucket WITH its index.
Every landed pack is complete with its index.
The renewer never skips a heartbeat while the holder moved: the sensor is honest.
No CAS lands from a deposed syncer after its successor restored.
A restore never refuses: the bucket is always restorable.

Mutations that must each find their loss
early ack (option B1) · no index gate (X1) · the checksum pass ticking nothing (run 3) · no rotation · the CAS before the packs (§4 reversed).

What the model found
Two rotation gaps in the code (X11, X12), fixed the same day with tests; then two gaps in itself, found by the liveness run.

Restore — a fresh pod from the bucket alone

Claim. Acquire the lease. Rotate the snapshot (seq + 1, same content; created empty if none) so any straggler's If-Match is stale before this pod serves a byte. Sweep every in-flight multipart upload: nothing of ours is in flight at this moment.
GET the snapshot; fetch the packs it names with .idx .bitmap .rev — one bounded fan-out across files and 8 MiB ranged chunks, 38-40 MiB of memory flat in the pack size, temporaries renamed only when every chunk has landed; write packed-refs and HEAD to EXACTLY the snapshot's refs; git fsck --connectivity-only; open the socket; serve.

A named pack the bucket lacks: re-read the snapshot once, then refuse loudly (exit 78, judged before the pod phase) rather than serve a repository git cannot see.
Measured: 40 GiB restored in 139 s from the delete (297 MiB/s), refs exactly the snapshot's, fsck clean, anon RSS 25 MiB; 1 GiB in 9 s.

The residual, and what the drills held

Told failed, but durable
A client that hangs up before the report is never noticed by the syncer: the batch completes and the bucket names the pushed tip while the client saw a failed push. Not a loss and not a corruption — the retry finds the ref already there — but a transition the argument carries in the other direction, kept as a required-fail probe in the model and seen on the wire (run 3; run 7's S7 and S8).

Acknowledged means durable, on real S3
runbx: 40 GiB push acknowledged in 1113 s, 641 parts, the CRC accepted at Complete, the snapshot at the tip; 40 of 40 pushes told ok were in the bucket across both takeover arms. S4, four kills placed INSIDE the multipart upload by watching list-multipart-uploads: told failed ⇒ the bucket unchanged; told ok ⇒ durable; every orphaned upload swept once the successor served.

The one write the pack directory has
git's own gc is off. The syncer repacks under its lock between batches, uploads the new pack, CASes a snapshot naming only it, then sweeps what no snapshot names — after a listing, after re-reading the snapshot, after a HEAD past a grace that outlives the longest upload. Measured cost: a full repack re-uploads the repository (every 24 pushes at the shipped threshold); geometric repack is the lever and needs a multi-pack bitmap.

What is NOT on this path

The runner and the door carry bytes and never write the bucket; the operator never reads it; the export and the derived dumb-protocol files are written after the report and are never a gate; a refused push's pack is never uploaded. Everything that can change what a restore sees goes through steps 3 and 4, under one lock, in one process.

durable path - - - the lease, on the control plane

The order of the steps is the argument. Packs go up before the snapshot names them, so a crash between the two leaves an unnamed object the sweep removes, never a name without an object; the snapshot is one object carrying every ref and the full pack list, so a reader never sees half a batch; the local ref transaction follows the CAS and the report follows the transaction, because per-ref `ok` reaches the client as it is emitted and a report interleaved with updates would acknowledge a subset the snapshot already holds in full. The pack listing that feeds step 3 requires the pack's `.idx`: git migrates a push's quarantine as `.keep`, `.pack`, `.rev`, `.idx`, in that order, so a neighbouring push's pack is on disk before its index for a moment, and a batch that listed it then would have named a pack whose index never followed — a restore of that snapshot refuses, exit 78, unrecoverable (X1, found by reading `tmp-objdir.c`, fixed by requiring the index). The CRC-64/NVME S3 judges at `CompleteMultipartUpload` is accumulated per part beside its PUT, so a 40 GiB pack is read once; the ~70 s pre-pass it replaced had, in its first shape, ticked no progress and let the lease go quiet inside a live push.

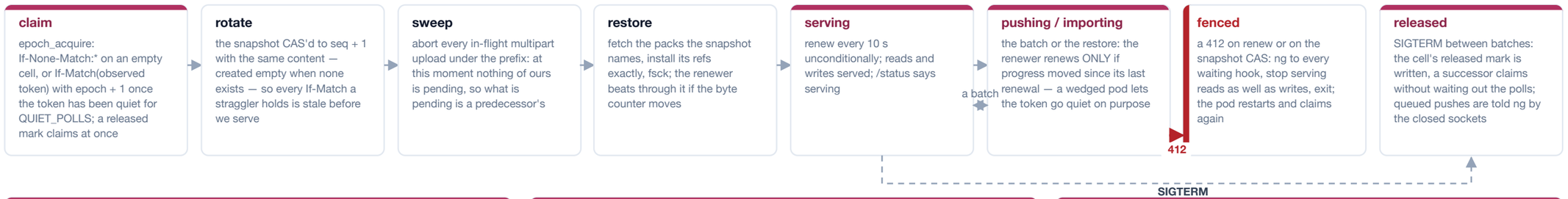
Five theorems hold over this path and five mutations must each lose one. `formal/ForgeSync.tla` models two syncers, two pushes, the batch at store-request granularity with a crash between any two steps, the renewer's byte-counter sensor kept distinct from real movement, a challenger, a successor's claim, rotation, sweep and restore, git's index-last migration, and a client that can hang up. The strict run holds 2,776,804 distinct states to depth 52 under a token-rank view and syncer/push symmetry: told `ok` implies the ref landed and the pack is in the bucket with its index; every landed pack is complete; the renewer never skips a heartbeat while the holder moved; no CAS lands from a deposed syncer after its successor restored; a restore never refuses. The mutations are the campaign's findings kept as regression tests — early acknowledgement, no index gate, a checksum pass that ticks nothing, no rotation, the ordering reversed — and each must find its loss or the gate fails. The model's first two strict runs refuted the code (page 4); its liveness run then refuted the model twice, a queued push left waiting by a clean release and a NoSuchUpload exit drawn from the crash budget, and the module moved.

One residual is kept rather than hidden, and the drills held the rest on real S3. A client that hangs up before the report is never noticed by the syncer, so the batch lands anyway and the bucket names a tip the client saw fail: not a loss and not a corruption — the retry finds the ref already there — but a transition the argument carries in the other direction, a required-fail probe in the model and seen on the wire in run 3 and in run 7's rollout leg. On runbx a 40 GiB push was acknowledged in 1113 s with 641 parts and the CRC accepted, and 40 of 40 pushes told `ok` across two takeover arms were in the bucket; four kills placed inside a multipart upload by watching `list-multipart-uploads` held told-failed ⇒ unchanged and told-ok ⇒ durable, and every orphaned upload was swept once the successor served. A fresh pod restores from the bucket alone: the snapshot's packs fetched by one bounded fan-out of 8 MiB ranged chunks at 38–40 MiB of memory flat in the pack size, refs installed exactly, `fsck --connectivity-only`, serve — 40 GiB in 139 s from the delete, fsck clean, 25 MiB of anon RSS.

The lease

One writer per repository is a mechanism, not a convention, and the mechanism is an epoch cell in S3 that every participant writes only by compare-and-swap. A holder renews it every ten seconds; a challenger judges the holder dead only after six consecutive polls saw the same token; a 412 on any CAS of ours is the fence, and the fence stops reads as well as writes, because a deposed server that kept answering `upload-pack` would serve stale refs forever. The subtle parts are the rotation that makes a straggler's writes stale before the successor serves, and a renewer that must stop renewing for a pod that is alive but wedged.

The holder's life — the lease cell is FlintTierEpoch's, and every write to it is a CAS



The challenger — a second syncer for the same repository

A roll against a wedged pod, a takeover of a dead node, or a hand-made challenger. It polls the cell every heartbeat and judges the holder dead ONLY by an unchanged token across six consecutive polls (60 s), then acquires with If-Match on that token and epoch + 1.

A renewing holder is undeposable, structurally: its own renew rotates the token, so the challenger never counts six. The scheduling axiom the model states and cannot discharge: the polls and the heartbeats share a period.

runbx, the seize arm CLOSED: a challenger beside a live 40 GiB restore for 398 s never claimed, the holder was never fenced, 24/24 pushes durable under the contention.

The renewer's gate — a sensor, and a sensor can lie

The renewer is its own task from the moment the claim lands. While the phase must progress (importing, pushing) it renews only if the operation's byte counter advanced since its last renewal; renewing for a wedged pod would trade a live pod losing its repository for a dead one keeping it forever.

Run 3: the checksum pre-pass over a 40 GiB pack (~70 s) did work and ticked nothing, so the token sat quiet for a whole takeover window inside a live push. The pass now ticks; then it was removed (D4: the CRC is accumulated as the parts are read).

The model keeps sensorMoved and realMoved apart and proves the sensor honest; the mutation that ticks nothing on the hash must lose.

What the model found in the rotation — X11, X12

X11: a takeover of a repository nobody had published skipped the rotation ("the first CAS's If-None-Match is the fence").

A straggler mid-batch on the old epoch landed its If-None-Match:* create after the successor served, and the successor's own first CAS was what 412'd. The rotation now creates the empty snapshot.

X12: a successor that died between its takeover CAS and its rotation came back through self-recognition, which skipped the rotation.

The straggler from the epoch before still held a valid If-Match. Every claim but a released cell's now rotates: only a releaser has proven it fenced itself before writing the mark. Both fixed with tests that fail against the old code.

Measured on the wire — the window, before and after

runbw, before the renewer task (window OPEN)

10 GiB push acknowledged in 262 s with the token silent 125 s; the cold restore 136 s with the token silent 141 s; a challenger claimed the live, importing pod 62 s after arriving; the two seized each other through epochs 4, 5, 6 until the challenger was removed at +392 s; 21 of 23 pushes acknowledged and every one in the bucket. The fence held and durability held; the cost was availability.

runbx, after (window CLOSED)

40 GiB push: 1113 s, the hook wait over 8 minutes, token silent ≤ 11 s; restore 139 s at 297 MiB/s, silent 11 s; the seize arm never claimed; run 5 at 10 GiB: silent ≤ 11 s, sampler blind 0 s.

A blind spot is not silence

The sampler's head-object hung 64 s on the CLI's own timeouts and read as 66 s of a quiet renewer. It now marks a sample blind, counts silence only across contiguous observations, and calls a blind spot past the bound inconclusive.

Numbers that shape the lease

```
heartbeat 10 s · QUIET_POLLS 6 · window 60 s
renew: If-Match(own token) → new token
claim: If-Match(quiet token), epoch + 1
release: released = true, token rotated
orphan grace: outlives the longest upload
```

A takeover costs one small CAS on the snapshot plus the restore. A clean roll costs one request: the successor claims the released cell at once. A crash costs the quiet polls, then the restore.

Nothing pins a straggler's writes: its packs are content-named and unnamed by any snapshot, so a straggler that completes an upload after the sweep leaves an object the next sweep removes — hygiene, not integrity, which is why no mutation run exists for the sweep.

In-flight multipart parts are billed until aborted: one interrupted 2 GiB push left 384 MiB of parts on runbw. The sweep runs at the claim and between batches, with no grace, because at those two moments nothing of ours is in flight.

How the lease relates to the other two planes

The lease cell is coordination — the control plane — and the fence it produces is what the durable path relies on: a 412 on any CAS of ours stops acknowledging AND stops serving reads. Pushes fail while S3 is unreachable; clones keep working for six heartbeats, then the holder withdraws readiness until a renewal lands — the lease and the process are kept (X13, found by the Continuity comparison and built the same day).

The operator does not read the cell. It takes the pod's word for its phase through /status; restart is not an operator decision — a fence exits 1, a refusal exits 78, and the kubelet restarts the pod.

Two servers for one repository is not a state the Deployment (Recreate) creates on its own; it takes a roll against a wedged pod, a lost node whose pod is still counted, or a hand. Every such case ends with the straggler's next CAS 412ing.

Open: X6 — a rollout during a long push SIGKILLs the batch at 30 s (measured, run 7: told failed, cleanly, the push lost); whether a roll should wait for the batch is the decision.

– – – control plane ——— durable path ▮ the fence / a hazard

A renewing holder is undeposable, structurally; the whole design is about when it stops renewing. The renewer is its own task from the moment the claim lands, so the restore, every batch and every export beat through it — the first shape renewed only inside a push, and then only between the upload's parts, which on runbw left a 10 GiB push's token silent for 125 s and its restore's for 141 s, inside a 60 s window; a challenger claimed the live, importing pod 62 s after arriving and the two seized each other through three epochs while every acknowledged push still reached the bucket. The fix is gated on purpose: while the phase must progress, the renewer renews only if the operation's byte counter advanced since its last renewal, because renewing for a wedged pod would trade a live pod losing its repository for a dead one keeping it forever, which is the case the quiet polls exist for. The counter is a sensor and a sensor can lie: run 3's checksum pre-pass did 70 s of work and ticked nothing. The model keeps the sensor and the real movement as separate variables and proves the sensor honest under the one quantitative axiom it states and cannot discharge — that the challenger's polls and the holder's heartbeats share a period. On runbx a challenger beside a live 40 GiB restore for 398 s never claimed, the holder was never fenced, and 24 of 24 pushes under the contention were durable.

The rotation is what a straggler cannot survive, and the model found two ways it was skipped. Every claim rotates the snapshot — the same content, a new etag — before the successor restores, so any `If-Match` a straggler still holds is stale before the successor serves a byte. The first shape exempted two cases, and the model's first two strict runs refuted both against the code the same day. A takeover of a repository nobody had published skipped the rotation on the theory that the first CAS's `If-None-Match` would be the fence; but a straggler mid-batch on the old epoch could land its create after the successor served, and then the successor's own first CAS was what 412'd, fencing it with its predecessor's push (X11: the rotation now creates the empty snapshot). And a successor that died between its takeover CAS and its rotation came back through self-recognition, which skipped the rotation because "our own previous process died with its writes" — while the straggler from the epoch before still held a valid `If-Match` (X12: every claim but a released cell's rotates, because only a releaser has proven it fenced itself before writing the mark). Both are unit tests that fail against the old code.

The cell is on the control plane and its consequence is on the durable path. The operator does not read it; the syncer's own phase says what the operator needs, and restart is not an operator decision — a fence exits 1, a refusal exits 78, the kubelet restarts the pod. Two servers for one repository is not a state the Deployment creates on its own; it takes a roll against a wedged pod, a lost node whose pod is still counted, or a hand, and every such case ends with the straggler's next CAS 412ing. The open item on this page is X6: a rollout during a long push SIGKILLs the batch at the 30 s grace, and run 7 measured exactly what that costs — the client told failed, the bucket unchanged, the retry converging, the orphaned upload swept — which is clean and loses the push; whether a roll should wait for the batch is the decision still to take.

The control plane

The control plane is the lite operator's shape, cut to one rung by copy-and-trim rather than import, and the lite gateway with a git arm. It answers three questions and only three: does this repository's pod exist and is it ready; may this token reach this repository; is anyone using it. It holds no S3 credential, reads no object, and sits in no transfer, which is why the runner could be swapped underneath it without a change to a line of it.

The operator — one FlintRepo becomes three objects and one idle rung

FlintRepo (CRD, short name fr)

`spec.bucket · spec.keyPrefix`
`spec.consumers.serviceAccounts`
`spec.branches: protected,`
`  pushers, agentPattern,`
`  mergeInto`
`spec.suspendAfterSecs`

keyPrefix is immutable: a new prefix is a new repository

reconcile

claim git/claim with this project's id (a foreign id: Refused, exit 78)
render a ConfigMap (the policy document pre-receive and the syncer both read), a headless Service, a Deployment of one pod — syncer + git container, Recreate, emptyDir, 30 s grace — and a NetworkPolicy admitting the door's pods only
apply the children; poll /status; compute the phase; write conditions

/status — the pod's own word

`{phase, activity.idleSecs, rpoClean,`
` epoch.lastRenewUnix, fenced, progress}`

phases: starting · importing · serving · pushing · draining · released. Ready = serving. Read only from the operators' namespace; the door refuses to proxy it.

the operator consumes phase and idleSecs; nothing reads lastRenewUnix, fenced, rpoClean or progress

the idle rung — replicas 0

no git traffic for suspendAfterSecs, judged on the polled document only (a poll failure holds forever) ⇒ replicas 0 and chart.us/idle-state; the emptyDir is gone, the repository is bucket objects and nothing else

X8: the activity clock counts pushes only — a fetch never reaches the syncer — so a clone-only repository is parked one threshold after its wake

X7: a failed poll with a Ready pod yields Starting, and the door waits on it

The door — admission, routing, wake

1 · authenticate

Basic x:<token> → TokenReview at the apiserver, the verdict cached ≤ 60 s by token hash; a refusal is cached, a transport failure is not. A clone is two to four requests, so 1,000 clones would otherwise be 3–4,000 reviews.

2 · authorise

spec.consumers must list the ServiceAccount; a credential-less peer gets 401 and is never dialled — authentication precedes the wake, so a peer cannot scale a parked repository up.

3 · route

one Service per FlintRepo, by name; the door's URL formula /git/<ns>/<repo>.git; X-Remote-User = system:serviceaccount:<ns>:<sa> is set from the verified token and cannot be smuggled past the allowlist.

4 · wake

a parked repository: arm chert.us/requested-at on the CR, wait on the CR — never the pod — up to 180 s (git clients do not retry a 503); the restore is the packs the snapshot names.

5 · bound

no bytes either way for 300 s cuts the request; a wake holds a slot ≤ 180 s. HTTPS at the door is a line in the diagram: nothing in-tree terminates it.

What the control plane never touches

The bucket. The operator holds no S3 credential and reads no object; the door holds none either.
A credential-free principal that could read the bucket would become a read-everything one for fields no decision reads — which is why C2 (the operator reading the bucket) was declined in the simplification note.
Restart. A fence exits 1, a refusal exits 78 judged before the pod phase, and the kubelet restarts the pod; the claim-against-a-healthy-holder case is the lease's, not the operator's.
The transfer. No control-plane request sits in a clone or a push; the TCP liveness probe reaches the status listener on its own task and cannot kill a batch.
A3 changed none of this: the chart's server.tag, the CRD, the door's URL formula and every rig were untouched by the runner.

Where the planes meet

control → data
the door's admission and routing decide which bytes flow at all; the wake creates the pod the bytes will reach; the NetworkPolicy is what makes X-Remote-User a principal.

control → durable
the lease cell's fence is what stops a deposed syncer from writing; the claim cell refuses a foreign project; the grace period decides whether a roll lands as a clean release or a SIGKILL mid-batch (X6).

durable → control
/status is written by the syncer; the operator believes it; rpoClean is always true by construction because nothing is acknowledged before the CAS.

data → durable
only through the hook: proc-receive is the one place the data plane hands anything to the writer, and it hands a request, never a credential.

Open on the control plane — recorded, not hidden

X6 grace: a rollout during a long push SIGKILLs the batch at 30 s. Measured on run 7: told failed, bucket unchanged, retry converges, orphans swept — clean, and the push is lost. Decide whether a roll waits for the batch.
X7 a failed /status poll with a Ready pod reads as Starting and takes a live repository out of rotation on a blind poll.
X8 the idle clock counts pushes only; clone-only repositories are parked and rewoken.
X9 readiness is Serving-only, so Pushing answers 503 and the headless DNS is withdrawn during a long push — unverified.
Not on the list because it is decided: KEDA's HTTP add-on could replace routing, wake and idle-to-zero with no flint code but not the auth, and the gateway exists (decision 12).

What the control plane has been through

The wake: an 11 s clone through the door on a pod the request created (F8's neighbour, EC2). The published artifact: the shipped chart could not install itself — it pinned the one image whose gateway rejected --git-only — found by a drill that installs the chart as a user would, which nothing had done; one server.tag now names both server images and the operator warns when the two references it is handed disagree (X10). The idle rung: the lite operator's ladder cut to one rung by copy-and-trim, not import, because ~800 of its lines are PVC and hibernate logic that is dead under an emptyDir cache.

– – – control plane data plane open item / hazard

The operator takes the pod's word, and the word is small. Reconcile claims `git/claim` with this project's id — a foreign id is `Refused`, exit 78, judged before the pod phase so a refusal does not read as a checkout in progress — renders the three objects and the policy ConfigMap both enforcers read, applies them, and polls `/status`. Of the document it gets back it consumes `phase` (Ready is `serving`) and `activity.idleSecs` (the idle rung); `lastRenewUnix`, `fenced`, `rpoClean` and `progress` are read by nothing and are diagnostic. The status port is reachable only from the operators' namespace and the door refuses to proxy it. The idle rung is lite's ladder with one step: no git traffic for `suspendAfterSecs`, judged on the polled document alone — a poll failure holds forever — means replicas 0 and an annotation, the emptyDir gone, the repository bucket objects and nothing else. The rung's clock is X8: it counts pushes only, because a fetch never reaches the syncer, so a clone-only repository is parked one threshold after its wake. X7 is its neighbour: a failed poll against a Ready pod reads as Starting and takes a live repository out of rotation on a blind poll.

The door does five things in order, and the order is the security property. Authenticate: the Basic password is the pod's token, verified by TokenReview at the apiserver and cached for at most 60 s by the token's hash, a refusal cached and a transport failure not — a clone is two to four requests, so 1,000 clones would otherwise be 3–4,000 reviews. Authorise: `spec.consumers` must list the ServiceAccount, and a credential-less peer gets 401 and is never dialled, so authentication precedes the wake and a peer cannot scale a parked repository up. Route: one headless Service per FlintRepo, by name; `X-Remote-User` is set from the verified token and cannot be smuggled past the door's allowlist. Wake: a parked repository is armed by annotating the CR, and the door waits on the CR, never the pod, for up to 180 s — git clients do not retry a 503, and a wake after an unclean death is the quiet polls plus a restore. Bound: no bytes either way for 300 s cuts the request. HTTPS at the door is a line in the diagram; nothing in-tree terminates it, and the token rides two cleartext hops.

Where the planes meet is one-directional at every joint. Control decides which bytes flow at all and makes the pod they will reach; control's lease produces the fence the durable path relies on, and its grace period decides whether a roll lands as a clean release or a SIGKILL mid-batch; the durable path writes `/status` and control believes it, with `rpoClean` true by construction because nothing is acknowledged before the CAS; and the data plane hands the durable path exactly one thing, through `proc-receive`, a request. The control plane has had its own campaign: the published-artifact drill found that the shipped chart could not install itself, because it pinned the one image whose gateway rejected `--git-only` — found by installing the chart as a user would, which nothing had done — and one `server.tag` now names both server images with the operator warning when the two references it is handed disagree. KEDA's HTTP add-on could replace routing, wake and idle-to-zero with no flint code, but not the auth, and the gateway exists; that is recorded as a decision, not an open question.

Boundaries and the record

Three boundaries, each stated with what it was measured to hold and what it does not; the policy that says who may move `main`, enforced twice; composition with the rest of the family on one bucket and the four conditions accepted by decision; and the ledger of every campaign behind this document. The last card is the edges: what has not been drilled, what has not been built, and what the model does not carry.

Three boundaries, and what each one actually holds

The credential boundary
AWS keys reach the syncer container by envFrom and nothing else: not the git container, not the door, not the operator, not an agent. The process that parses untrusted HTTP cannot write the bucket.

The principal boundary
REMOTE_USER is the door's X-Remote-User, set from a verified TokenReview. Reached directly, the git port believes the header — so the operator renders a NetworkPolicy admitting the door's pods only, and reaching the port is the authorisation where it is not rendered.
Measured (run-rights.sh, Cilium): with the policy in place a forged header cannot open 8080; with it deleted the same header merges into main. 17 legs, green, twice. kind's default CNI enforces no policy at all.

The wire
two cleartext hops as built; the pod's audience-bound token rides both as a Basic password; server-to-S3 is TLS.

Policy — who moves main, enforced twice

```
protected: [main, release/*]
pushers: {main: [release-bot]}
agentPattern: agent/*
mergeInto: {main: [agent-runner]}
```

pre-receive names the rule at the edge; the syncer enforces the same document in its judge step, so a missing hook does not open the repository.
Merge is a push: HEAD:refs/for/main runs merge-tree --write-tree in the syncer, packs what it created, and the batch CAS carries main; refs/for is never stored. One path to S3, structurally.
Agents never hold an S3 credential; humans use the gateway's bearer today. The Rights axis of the radar has three of seven cells on the wire: passthrough per-pod readOnly, forge per-SA policy and the NetworkPolicy boundary.

Composition on one bucket — with lean and passthrough

The export publishes main as a lean workspace after the report; lean and passthrough readers mount it read-only with no forge code in them. Disjoint prefixes and disjoint lease cells.

Accepted conditions (C1–C5, 2026-09-05)
A1 forge and lean on one prefix do not arbitrate: detection shipped (flint_store::layout), prevention belongs to whatever assigns prefixes.
A2 a nested export prefix is admitted; the sweep lists only pack/ and bundles/, so nothing is destroyed.
A3 a foreign write into the export is neither seen nor repaired; A4 a reader with no manifest takes the foreign bytes. Declined: it needs a second writer on the export prefix, a misconfiguration.
Fixed: C2 a second writer on the export prefix wedged the repository; C4 the manifest-checking readers refuse.

The record — every campaign behind this document, 2026-09-04 → 2026-09-05

falsifiers F1–F11, EC2	acknowledged-means-durable under 8 mid-push kills; two pushes to one ref; a merge surviving a cold restore; the fence deposing a straggler within one heartbeat; a byte-identical restore and a dumb clone with the server down; protected main (found a fresh repository unable to create main); idle-to-zero with an 11 s clone through the door on a pod the request created; the storm (5.7 MiB vs 40,409 MiB); the export; the sweep; an S3 outage — clones served, pushes refused.
composition C1–C5, MinIO rig	two fixed (a second writer wedging the repository; readers taking foreign bytes), three accepted by decision, each leg reporting KNOWN and turning STALE if its condition stops reproducing.
rights, kind + Cilium	17 legs: a reader refused a direct push and a merge, a writer merging, a forged header overridden at the door and believed at the port without the policy.
latency rig, toxiproxy	a push costs 5.1 round trips + 225 ms with the sibling fan-out and 7.1 without; a 33-file restore 14.3 round trips against 38.6 serial; the renewer through an 8.8 s restore: 7 rotations, longest silence 1.2 s.
runbw, 3 × i4i.xlarge, real S3	10 GiB composed as 161 parts, CRC accepted; the token silent 125 s in the push and 141 s in the restore — the window OPEN; a challenger seized a live restoring pod at 62 s; every acknowledged push in the bucket; one interrupted upload left 384 MiB of billed parts.
runbx, the confirmation	40 GiB pushed in 1113 s and restored in 139 s with the token never silent past 11 s; the seize arm closed against a real challenger; told-ok ⇒ durable 40/40; three front-layer defects found in nginx and fcgiwrap — the case for A3 — and a rig sampler that lied.
runby, the runner's acceptance	S5–S10 through flint-forge-gitcgi: a 70 s client stall acknowledged; five pushes and eight clones served concurrently; 48 keepalives across a 232 s hook wait with a 5.8 s longest gap; a rollout mid-push told failed cleanly. The nginx control failed exactly those legs: 502 at 60 s, four workers, one burst, four upload-packs.
the model, ForgeSync.tla	2,776,804 states strict; five mutations that must lose; a required-fail probe; two code defects found in the rotation and two model defects found by the liveness run, all the same day.

Not drilled, not built — the edges as they stand

Agents in another cluster (the door is reachable from one). Production: nothing here has served a team; every number is a rig's. A bought forge on a flint volume: the buy-option control ran on a local disk, so "git over NFS is slow" is unmeasured. Read replicas: one pod serves each repository, and the storm lever is the bucket, not a second server. A replayable log: the snapshot is state, not history; provenance is git's own.
Known and kept: told-failed-but-durable (the retry converges); a rollout mid-push loses the push (X6); the export is a mirror nobody repairs (A3); a reader with no manifest takes foreign bytes (A4); the 30 s grace, the push-only idle clock (X8) and the blind-poll Starting (X7) are open; geometric repack needs a multi-pack bitmap before the re-upload-everything cost of a full repack goes away.
What the model does not carry: time (the six-quiet-polls window is a stated axiom), sizes, the door, the operator, and git's own correctness.

forge boundary / accepted condition / edge

The principal boundary is a NetworkPolicy, and it was shown to be one. `REMOTE_USER` is what `pre-receive` and the syncer read as the principal, and it arrives as the door's `X-Remote-User`, set from a verified TokenReview. Anything that reaches the pod's git port directly can set that header itself, and both enforcers would believe it; what stands in the way is a headless Service with no external address, the tenant's namespace, and a NetworkPolicy the operator renders — only when told where the door runs — admitting the door's pods alone. On a kind cluster with Cilium, because kind's default CNI enforces no policy at all, a forged header sent to the door was overridden, a forged header sent to the port with the policy in place could not open it, and the same header with the policy deleted merged into `main`: 17 legs, green, twice. That pair is the evidence that the header means something only behind the policy; where the policy is not rendered, reaching the port is the authorisation, which is defensible on a single-tenant cluster and is not on a shared one. The credential boundary is simpler and stronger: the bucket key reaches the syncer container by `envFrom` and no other process. Merge is a push — `HEAD:refs/for/main` runs `merge-tree --write-tree` in the syncer, packs what it created, and the batch CAS carries `main` — so there is one path to S3, structurally rather than by convention, and the policy that names who may propose it is enforced at the edge and again in the judge step.

Composition on one bucket is a convention across products and a mechanism within one, deliberately. The export publishes `main` as a valid lean workspace after the report, by the shipped `flint-sync barrier`, and lean and passthrough readers mount it read-only with no forge code in them. Five composition drills on a MinIO rig found five things; two were fixed (a second writer on the export prefix wedged the repository; manifest-checking readers now refuse foreign bytes) and three stand by decision — forge and lean on one prefix do not arbitrate but the condition is now audible; a nested export prefix is admitted but the sweep touches nothing outside its own two directories; a foreign write into the export is neither seen nor repaired, and a reader with no manifest takes it, because closing that needs a second writer on the export prefix, which is a misconfiguration. Each accepted leg reports `KNOWN` and turns `STALE` if its condition stops reproducing, so a record that has quietly become wrong is also something a human has to look at.

The record is what the numbers on every other page rest on, and the edges are what they leave out. Eleven falsifiers on EC2; the composition, rights and published-artifact drills on kind; the latency rig with injected round trips; three campaigns on real S3 — runbw, which opened the takeover window on the wire; runbx, which closed it and found the case for the runner; runby, which accepted the runner against a control that failed exactly the legs it had to — and the model with its mutations. Not drilled: agents in another cluster, production, a bought forge on a flint volume. Not built: read replicas, a replayable log, a UI, review. Kept: told-failed-but-durable, a rollout mid-push losing the push, the export as a mirror nobody repairs, the push-only idle clock, the blind-poll Starting, and a full repack that re-uploads the repository every 24 pushes until geometric repack gets a multi-pack bitmap.

Prior art and the verdict

The question asked of this document was how innovative forge is in capturing the git-and-S3 integration. The honest answer needs the table: five systems that put git's durable state somewhere other than the serving host's disk, one of them published seventeen days before forge's design, beside forge on the dimensions that decide correctness and cost. What is convergent is convergent; what is forge's own is smaller than the whole and is stated as such.

dimension	GitHub Spokes	GitLab Gitaly	AWS CodeCommit	Stemma / JGit DFS	Continuity · walgit	flint-forge
durable substrate	three replicas on local disks in three racks, replicated at the application level	local SSD per Gitaly node, Praefect replicating; object storage for LFS and artifacts only	packs in S3, metadata in DynamoDB; managed (closed 2024-07, GA again 2025-11-24)	packs as blobs and refs as rows in a transactional store (AtlasDB; Google: Bigtable)	a write-ahead log of per-push objects in S3/GCS plus a CAS'd index or manifest	**content-named packs in S3 plus ONE CAS'd snapshot; nothing else is trusted**
what serves git	stock git per replica, behind a proxy	stock git inside Gitaly, over gRPC	proprietary	JGit, in Java, on a DFS abstraction	stock git on disk (Continuity); walgit: its own receive-pack in Rust, stock git for upload-pack, repack and bundles	**stock git — receive-pack, upload-pack, hooks — behind a 382-line CGI runner; flint writes no git internals**
the write, and the ack	a git transaction on ≥ 2 of 3 replicas by three-phase commit; ok after that	a transaction on the primary's disk, then replicated; ok after the primary	a managed write; ok after it	a database transaction over refs and pack ids; ok on commit	one WAL entry — the packfile and its ref transaction — then the index CAS; "never acknowledge a push until it has been fully persisted"	**every complete pack uploaded, ONE snapshot CAS, the local ref transaction, THEN ok — carried by proc-receive, which waits on the syncer**
writer coordination	a proxy per push; consensus among the replicas	Praefect elects a primary per repository	the service	database transactions	the index/manifest CAS is the consensus: no leader, no election, any host may be primary	**one writer per repository under an S3 epoch lease with a progress-gated renewer; every claim rotates the snapshot so a straggler's If-Match is stale first**
disk, idle, restore	the repository itself, three times; always on	the repository itself; always on	none visible; managed	a DFS block cache; always on	a warm cache materialised from the WAL; idle replicas evicted and rematerialised on the next fetch	**an emptyDir cache restored from the snapshot at every start (40 GiB in 139 s); one pod per repository, parked at replicas 0, woken by the door**
reads and storms	three replicas serve reads	Praefect distributes reads	the service	JGit caches	read replicas kept current by conditional GET; linear to 100 replicas; 120 pushes/s on S3 Standard	**one pod; the lever is bundle URIs on the bucket — 5.7 MiB against 40,409 MiB of egress for 1,000 clones**
identity and policy	the platform's	the platform's	IAM	the platform's	token or OIDC; per-repository push policy (walgit)	**the pod's own ServiceAccount token, by TokenReview; consumers and a branch policy per principal, enforced twice; merge is a push**
verification	production at GitHub scale	production at GitLab scale	production since 2015	production at Palantir	production at Cursor; walgit: a seeded fault-injection simulation (crash, partition, stale, lost response), ~2.4k stars	**a TLA+ model with mutations, eleven falsifiers, three campaigns on real S3, a control arm — and no production**

The verdict

Convergent, and independent: S3 as the only durable state, disk as a cache, one CAS'd pointer and stock git is a shape Cursor published on 2026-08-18 and forge reached on 2026-09-04, with CodeCommit (2015) and Stemma (2017) before both on other stores. Forge's own: the syncer as the sole writer behind stock receive-pack with the acknowledgement carried by proc-receive; the progress-gated lease with takeover rotation; a per-repository Kubernetes control plane with pod identity and idle-to-zero; the legible export; and the model-and-drill record. Not forge's: read replicas, a replayable log, a UI, review.

The shape is convergent, and forge arrived at it independently in the same month as its nearest neighbour. "S3 as the only durable state, the on-disk repository a cache, one CAS'd pointer as the transaction boundary, stock git doing every git operation" is what Cursor published as Continuity on 2026-08-18 — a write-ahead log of per-push packfiles and reference transactions in S3, an index object rewritten by compare-and-swap, hosts that are stateless and any of which may be primary, read replicas kept current by conditional GET, 120 pushes/s measured on S3 Standard — and what `walgit`, an open-source Rust server on that design, ships with bundle-uri, LFS and per-repository push policy. Forge's design is dated 2026-09-04 and cites neither, because its author had not seen them; the design's own prior-art section named AWS CodeCommit, which stored packs in S3 and metadata in DynamoDB from 2015, closed to new customers in July 2024 and returned to general availability on 2025-11-24 — a correction this document carries back into the design. Behind all of them is Palantir's Stemma on JGit's `DfsRepository`, packfiles as blobs and refs as rows in a transactional store, and Google's Bigtable-backed JGit before it. The S3 primitive that makes the pointer possible without a database is itself recent: conditional writes with `If-None-Match` arrived in August 2024 and `If-Match` on the ETag in November 2024, and the flint-store abstraction forge builds on was written against them for lean before forge existed. So the claim "a git server whose only durable state is S3" is not forge's; the claim is that forge is a correct and measured instance of a shape several teams reached in the same eighteen months, one of them in the same month.

What is forge's own is in the joints, not the shape. Four things in the table have no counterpart in the row above them. The writer is stock `receive-pack` with the syncer as its only client and the acknowledgement carried by `proc-receive` waiting on it, so "told `ok`" is the syncer's report and not the server's disk — Continuity states the same guarantee and does not say how it is carried; walgit gets it by implementing receive-pack itself, in Rust, and leaving upload-pack, repack and bundles to stock git. Coordination is a single-writer lease with a progress-gated renewer and a rotation on every claim, chosen over "any host may CAS" because at fleet rates every self-inflicted 412 would be a full restore, and refined by a model that found two rotation gaps in the code the day it was written. The control plane is Kubernetes-native per repository — the pod's own ServiceAccount token as the credential, one pod parked at replicas 0 by an operator and woken by a door that authenticates before it wakes, a branch policy per principal enforced at the edge and in the writer — where the neighbours are hosted services with their own identity. And the legible export publishes the repository as a lean workspace the rest of the family mounts with no git in it, which nothing in the table attempts. A fifth thing is method rather than architecture: a TLA+ model whose mutations are the campaign's findings, eleven falsifiers, three campaigns on real S3 and a control arm that must fail, all in the repository, where Continuity's verification is production and walgit's is a seeded fault-injection simulation.

The verdict, stated so that it can be wrong. Innovative in the integration's joints — how the acknowledgement is carried, how one writer is kept and deposed, how the server is placed and woken, how the repository is re-published for readers that are not git — and convergent in its shape, which is now the shape of the field. **The comparison this page asked for ran the same day** (`forge/e2e/walgit/`): forge won the 1 GiB push and drew the clone storm, and lost the write path at rate by an order of magnitude from one joint, the full repack every 24 packs, which uploaded 33 times the bytes pushed and held one push for 816 s; The tiers and the window were built the next day and the re-match run: the worst push fell from 816 s to 0.83 s, the rate rose to 14.1 pushes per second against walgit's 10.3, and walgit still uploads a third of the bytes — so forge keeps its place on the wire, with the bytes as the next number. The next honest measurement remains a bought forge on a flint volume.